Frecuencímetro digital sobre FPGA

Laboratorio 1 — Sistemas Electrónicos de Instrumentación

Pedro Villar | 20 de septiembre de 2026

1. Introducción

Se implementó un frecuencímetro digital de cuatro dígitos en la placa **Boolean Board** (FPGA Xilinx Spartan-7).

Usamos el *método de medición directa*: se abre una ventana de tiempo conocida y se cuentan los flancos de subida de la señal de entrada que caen dentro de esa ventana.

$$f_x = \frac{N_{\text{pulsos}}}{T_{\text{ventana}}}, \qquad \text{con } T_{\text{ventana}} = 1 \text{ s} \;\Rightarrow\; f_x = N_{\text{pulsos}} \text{ [Hz]}.$$

Es decir, con una ventana de exactamente un segundo la cuenta acumulada *es* la frecuencia en hertz. El resultado se muestra en el display de la placa, con un rango de 0 a 9999 Hz.

2. Arquitectura del diseño

El sistema se dividió en módulos independientes, todos instanciados dentro del módulo `Frequency_meter.sv`, que hace de *top-level*. La Figura 1 muestra cómo quedan conectados.

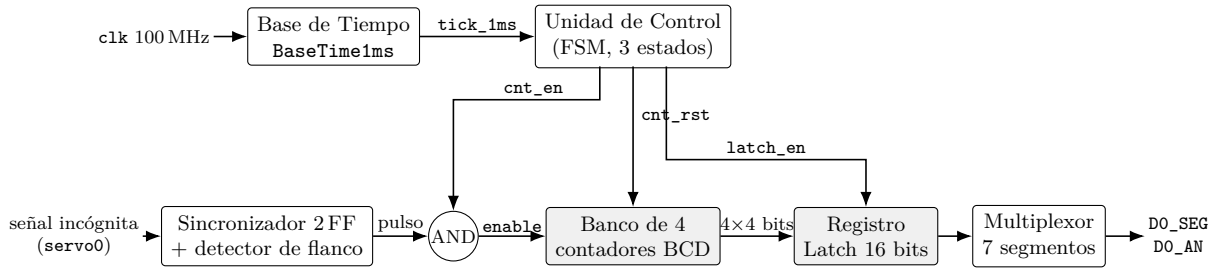


Figura 1: Diagrama de bloques del frecuencímetro. En gris, los bloques escritos; el multiplexor de 7 segmentos nos lo dió el profesor.

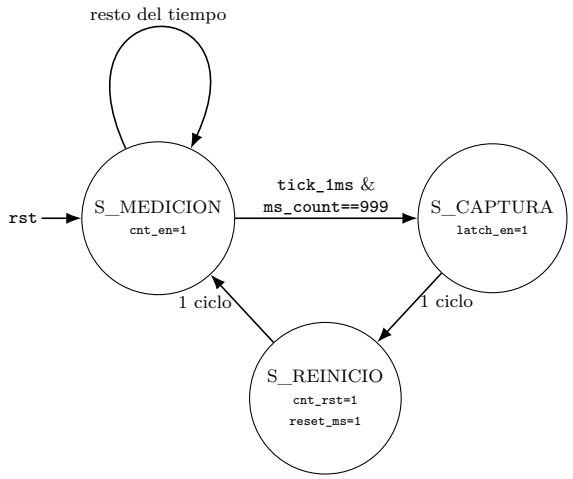
Decisiones de diseño

- **Base de tiempo en dos etapas.** En lugar de un único contador de 0 a 99.999.999 (27 bits), se reutilizó el módulo `BaseTime1ms` provisto, que divide los 100 MHz y emite un pulso cada milisegundo, y se cuentan 1000 de esos pulsos dentro de la propia FSM. Quedan dos contadores chicos (17 y 10 bits) en vez de uno grande, y la ventana queda expresada en milisegundos.
- **Un solo dominio de reloj.** Todo el diseño es síncrono al reloj de 100 MHz. La señal incógnita *no* se usa como reloj: se la muestrea como un dato más. Esto evita generar relojes derivados.
- **Sincronizador y detector de flanco.** La señal de entrada es asíncrona respecto del reloj, así que pasa primero por dos flip-flops en cascada y después por un tercero que guarda el valor anterior. Con `freq_sync_1 & ~freq_prev` se obtiene un pulso de un único ciclo por cada flanco de subida, lo que garantiza que cada flanco incremente el contador exactamente una vez, sin importar el ancho del pulso de entrada.
- **La ventana es una compuerta AND.** El pulso de flanco se combina con `cnt_en` de la FSM (`count_pulse = freq_edge & cnt_en`). Cuando la FSM cierra la ventana, el contador simplemente deja de recibir pulsos.
- **Contadores BCD en cascada en lugar de binario.** Cada dígito cuenta de 0 a 9 y genera un `carry_out` de un ciclo que habilita al dígito siguiente. Así el resultado ya sale separado en dígitos decimales de 4 bits y se conecta directo al decodificador, sin necesidad de hacer divisiones sucesivas por 10 en hardware.

- **Registro latch para estabilizar la lectura.** Sin él, el display mostraría el conteo creciendo durante todo el segundo. El banco de flip-flops D captura el valor solo cuando la FSM activa `latch_en`, de modo que en pantalla siempre se ve la última medición completa y el número se actualiza una vez por segundo.

Unidad de control (FSM)

La unidad de control es una máquina de estados con un contador interno `ms_count` que acumula los `tick_1ms` mientras dura la medición.



Estado	`cnt_en`	`latch_en`	`cnt_rst`
S_MEDICION	1	0	0
S_CAPTURA	0	1	0
S_REINICIO	0	0	1

Salidas de la unidad de control.

Figura 2: Diagrama de estados de `FSM_Control`.

El ciclo de trabajo es el siguiente. En **S_MEDICION** los contadores están habilitados y la FSM espera a que lleguen 1000 ticks de 1 ms. En **S_CAPTURA**, que dura un único ciclo de reloj, se levanta `latch_en` y el registro guarda el valor alcanzado. En **S_REINICIO**, también de un ciclo, se borran los contadores BCD y el contador de milisegundos, y la máquina vuelve a medir. La lógica de próximo estado y de salidas está escrita en un bloque `always_comb` con valores por defecto asignados al principio, para que la síntesis no infiera latches no deseados.

3. Resultados y síntesis

El diseño se sintetizó, se implementó y se cargó en la Boolean Board. Para la prueba se inyectó por el pin `servo0` una señal desde un generador de funciones GW Instek AFG-2005, de 1,700 V_{pp} con un offset de 0,90 V, y se comparó en paralelo la frecuencia ajustada en el generador con lo que mostraban los displays de la placa.

Tabla 1: Frecuencia de prueba inyectada y lectura obtenida en los displays.

Frecuencia inyectada (generador)	Lectura en displays	Observación
7,894 kHz	7894 Hz	El dígito de los millares no aparece en la foto. Justo la camara no logro agarrar bien la foto, pero en la placa la lectura se veía completa.



(a) Generador: FREQ = 7,8940 kHz



(b) Display de la Boolean Board

Figura 3: Medición sobre la placa. A la izquierda, la frecuencia ajustada en el generador de funciones; a la derecha, el valor contado por el frecuencímetro en el display de siete segmentos.

Costo espacial del diseño. La Tabla 2 resume los recursos que reportó Vivado en el *Utilization Report* después de la implementación.

Tabla 2: Recursos utilizados. Vivado 2024.2, post-implementación, dispositivo `xc7s50csga324-1`.

Recurso	Usados	Disponibles	Utilización
LUTs (Slice LUTs)	51	32.600	0,16 %
Flip-Flops (Slice Registers)	105	65.200	0,16 %
Bonded IOB (pines)	15	210	7,14 %

4. Conclusión

Lo que más me costó al principio fue asimilar que acá no estoy programando, sino describiendo. La secuencialidad del código, que en software es la base fundamental de todo lo que uno hace, deja de existir: no hay una línea que se ejecute después de la otra, sino bloques que quedan instanciados y funcionando en paralelo. Eso complica además la forma de depurar, porque las cosas no se disparan como en un programa y uno pierde la referencia de por dónde va la ejecución. De todas formas, me parece que el aprendizaje más importante no pasa por ahí, sino por ver la diferencia entre ejecutar una tarea sobre un micro ya armado y ejecutarla directamente sobre compuertas. Corriendo sobre la FPGA obviamente se gana velocidad, pero se complica bastante más, porque no estamos trabajando sobre un sistema operativo que nos resuelva todo lo que hay por debajo. Ahí es donde me parece que está lo más valioso de este trabajo: entender el *trade-off* entre resolver algo en hardware o resolverlo en firmware.